

A Compact Post-quantum Strong Designated Verifier Signature Scheme from Isogenies

Farzin Renan

Middle East Technical University, Ankara, Turkey

farzin.renan@gmail.com

Abstract

Digital signatures are fundamental cryptographic tools that provide authentication and integrity in digital communications. However, privacy-sensitive applications—such as e-voting and digital cash—require more restrictive verification models to ensure confidentiality and control. Strong Designated Verifier Signature (SDVS) schemes address this need by enabling the signer to designate a specific verifier, ensuring that only this party can validate the signature. Existing SDVS constructions are primarily based on number-theoretic assumptions and are therefore vulnerable to quantum attacks. Although post-quantum alternatives—particularly those based on lattices—have been proposed, they often entail large key and signature sizes.

In this work, we present CSI-SDVS, a novel isogeny-based SDVS scheme that offers a compact, quantum-resistant alternative to existing SDVS constructions. The scheme leverages the ideal class group action on $\mathbb{F}_p$ -isomorphism classes of supersingular elliptic curves and is founded on the hardness of the Multi-Target Group Action Inverse Problem (MT-GAIP). CSI-SDVS achieves strong security guarantees—Strong Unforgeability under Chosen-Message Attacks (SUF-CMA), Non-Transferability (NT), and Privacy of Signer’s Identity (PSI)—in the random oracle model, thereby making it among the most compact PQC-based SDVS schemes and the only post-quantum secure construction based on isogenies.

Keywords: Digital Signature; Strong Designated Verifier; Post-quantum Cryptography; Isogeny-based Cryptography

1 Introduction

In many cryptographic applications, digital signatures play a central role by ensuring message authenticity and protecting against tampering. Yet, their default public verifiability model may not align with scenarios that demand selective disclosure or restricted verification. In particular, contexts involving sensitive or confidential data—such as personalized medical documents, legal contracts, or anonymous transactions—necessitate a finer level of control over who can verify the legitimacy of a signature. These limitations have motivated the exploration of alternative signature paradigms that can embed verification constraints directly into the signing process.

To address these concerns, Jakobsson et al. [1] and Chaum [2] independently introduced the notion of *Designated Verifier Signatures* (DVS) in 1996. In a DVS scheme, only a specific verifier

is capable of validating the authenticity of a signature, while any third party remains unconvinced. This is achieved by enabling the designated verifier to simulate signatures that are computationally indistinguishable from authentic ones, thereby preserving ambiguity about their origin. Even if a valid-looking signature is disclosed, only the designated verifier can be confident of its validity. This crucial feature—referred to as *non-transferability*—ensures that the verification outcome is non-provable to others.

Building upon this idea, the concept of *Strong Designated Verifier Signatures* (SDVS) later emerged as an extension of the standard DVS framework, offering enhanced privacy guarantees. Although briefly mentioned in the appendix of Jakobsson et al.’s foundational work [1], a formal definition and construction were first provided by Laguillaumie et al. [3] in 2004. The key advancement in SDVS lies in the protection of the signer’s identity against any third party, including passive observers, even in the presence of multiple potential signers. In particular, SDVS schemes satisfy the *Privacy of Signer’s Identity* property: given a valid signature and two candidate signing public keys, it is computationally infeasible for anyone other than the designated verifier to determine which key was used. This is accomplished by incorporating the designated verifier’s secret key into the verification algorithm, rendering the process inherently non-transferable and private.

1.1 Related Work

Strong designated verifier signature schemes have been extensively studied in both classical and post-quantum cryptographic settings.

Classical Constructions. Independent of the foundational work by Laguillaumie et al. [3], Saeednia et al. [4] proposed an efficient SDVS scheme by adapting the Schnorr signature and incorporating the designated verifier’s secret key into the verification process. Subsequently, Huang et al. [5] and Kang et al. [6] introduced identity-based SDVS (IBSDVS) schemes and proved their unforgeability under the Bilinear Diffie–Hellman (BDH) assumption. However, most classical SDVS schemes rely on number-theoretic hardness assumptions—such as integer factorization and discrete logarithms—which are rendered insecure in the presence of quantum adversaries, as Shor’s algorithm [7] can efficiently solve these problems.

Post-Quantum Constructions. In response to the vulnerabilities of classical assumptions, post-quantum SDVS schemes have been developed based on quantum-resistant foundations. Code-based SDVS constructions were proposed by Assar [8], Ren et al. [9], and Shooshtari et al. [10], leveraging the hardness of the bounded syndrome decoding problem in coding theory [11]. While these schemes are believed to be quantum-resistant, it was shown in [12] that the constructions by Ren et al. and Shooshtari et al. fail to satisfy the critical non-transferability property, which is central to the SDVS notion. One of the earliest lattice-based SDVS constructions was introduced by Wang et al. [14], employing *preimage sampleable functions* (PSFs) and Bonsai Trees for trapdoor delegation. Their work builds on the influential framework of Gentry et al. [13], which introduced PSFs as a core primitive in lattice-based signatures. Although the construction offers provable security in the random oracle model, it suffers from efficiency issues due to large key and signature sizes. Later, in 2016, Noh et al. [15] proposed a scheme in the standard model based on an LWE-based encryption scheme and a lattice-based chameleon hash function, albeit at the cost of significant computational overhead. To improve practicality, Cai et al. [16] introduced a more efficient scheme based on the Ring-SIS assumption, using rejection sampling to reduce signature size while maintaining security. More recently, Zhang et al. [17] proposed an SDVS construction combining SIS and LWE assumptions, further contributing to the development of quantum-secure SDVS protocols. However,

lattice-based schemes generally incur key and signature sizes with complexity $\mathcal{O}(\lambda^2)$ [14, 15, 16, 17], which may limit their suitability for resource-constrained environments.

It is worth mentioning that Sun et al. [18] proposed an isogeny-based SDVS scheme built upon the Supersingular Isogeny Diffie–Hellman (SIDH) protocol [19]. However, this construction is no longer considered secure due to several successful cryptanalytic attacks [20, 21, 22] against SIDH. Furthermore, the scheme employed deterministic signing, which may introduce privacy and replayability concerns in certain scenarios.

1.2 Our Contribution

Our main contributions are as follows:

- We present CSI-SDVS, an efficient post-quantum SDVS scheme that builds on ideal class group action on $\mathbb{F}_p$ -isomorphism classes of supersingular elliptic curves. The security of our scheme is based on the hardness of the Multi-Target Group Action Inverse Problem (MT-GAIP) [24], and its design draws inspiration from the CSIDH [23] framework and the cyclic structure of the ideal class group as described in CSI-FiSh [24].
- The scheme provides strong security guarantees, including Strong Unforgeability under Chosen-Message Attacks (SUF-CMA), Non-Transferability (NT), and Privacy of Signer’s Identity (PSI), making it well-suited for applications requiring strict confidentiality and verifier-specific authentication.
- Our isogeny-based construction naturally yields compact keys and signatures—a critical advantage over lattice-based SDVS schemes that inherently suffer from large key and signature sizes.
- To the best of our knowledge, this is the only SDVS scheme based on isogenies that offers post-quantum security, contributing a new design within the scope of SDVS schemes.

Acknowledgements. I would like to express my gratitude to Sarah Arpin, Jason LeGrow, Frederik Vercauteren, and the Isocrypt Study Group for their constructive feedback, which helped improve this paper. I am also especially grateful to Péter Kutas for his insightful discussions during the early stages of the draft.

Outline. In the next section, we recall the formal definition of an SDVS scheme and provide the associated security requirements that such a scheme must satisfy. We then present a brief overview of the necessary mathematical prerequisites and describe the underlying group action-based hardness assumptions upon which our construction relies. In Section 3, we introduce our proposed SDVS scheme, CSI-SDVS. Section 4 analyzes the efficiency of our construction, and Section 5 provides a formal security proof.

2 Preliminaries

Notation. We denote the security parameter by λ . For a finite set $\mathbb{X}$, the notation $x \xleftarrow{\$} \mathbb{X}$ indicates that x is sampled uniformly at random from $\mathbb{X}$. A probabilistic polynomial-time (PPT) algorithm A that outputs a value x on input y is denoted by $x \xleftarrow{\$} A(y)$. In contrast, for a deterministic polynomial-time (DPT) algorithm, we write $x := A(y)$. A function $\epsilon : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is said to be *negligible* if for every positive polynomial $p(\cdot)$, there exists a threshold $n_0 \in \mathbb{N}$ such that for all $n \geq n_0$, it holds that $\epsilon(n) < \frac{1}{p(n)}$. The concatenation of two bit strings s_1 and s_2 is written as

$s_1 \| s_2$. For any positive integer n , we define $\mathbb{Z}_n := \mathbb{Z}/n\mathbb{Z}$. We use the notation $\forall i \in [T]$ to denote "for all $i \in \{1, 2, \dots, T\}$ ".

2.1 Strong Designated Verifier Signature (SDVS) Scheme

An SDVS scheme involves two distinct parties: a signer S and a designated verifier V , and is defined over a message space $\mathcal{M}$ and a signature space Σ . In such a scheme, the signer S , holding a secret key sk_S , generates a signature $\sigma \in \Sigma$ on a message $m \in \mathcal{M}$ using its secret key and the public key of the designated verifier. The resulting signature σ can be verified only by V using its secret key sk_V , ensuring that no third party can verify the signature or be convinced of its validity. This restriction preserves the signer's privacy by eliminating public verifiability while maintaining correctness for the designated verifier. Moreover, the designated verifier is equipped with a simulation algorithm Simul that can generate a simulated signature σ' on any message $m' \in \mathcal{M}$ using sk_V and the signer's public key, thereby preventing any verifiable evidence of the signer's involvement. The formal syntax of an SDVS scheme is presented below, following the framework introduced in [18].

Definition 2.1 (SDVS Scheme). *A strong designated verifier signature scheme is defined as a tuple of six polynomial-time algorithms:*

$\text{Setup}(1^\lambda) \xrightarrow{\S} \text{pp}$: A PPT algorithm that, given a security parameter λ , outputs the public parameters pp .

$\text{SigKeyGen}(\text{pp}) \xrightarrow{\S} (sk_S, pk_S)$: A PPT algorithm that, on input pp , generates a secret/public key pair (sk_S, pk_S) for the signer S .

$\text{VerKeyGen}(\text{pp}) \xrightarrow{\S} (sk_V, pk_V)$: A PPT algorithm that, on input pp , generates a secret/public key pair (sk_V, pk_V) for the designated verifier V .

$\text{Sign}(\text{pp}, sk_S, pk_V, m) \xrightarrow{\S} \sigma$: A PPT (or DPT) algorithm that, given the public parameters pp , the signer's secret key sk_S , the public key pk_V of the designated verifier, and a message $m \in \mathcal{M}$, outputs a real designated verifier signature $\sigma \in \Sigma$.

$\text{Verify}(\text{pp}, sk_V, pk_S, m, \sigma) \rightarrow 0/1$: A DPT algorithm that, given the public parameters pp , the designated verifier's secret key sk_V , the signer's public key pk_S , a message $m \in \mathcal{M}$, and a signature $\sigma \in \Sigma$, outputs a decision bit $b \in \{0, 1\}$, where $b = 1$ indicates acceptance and $b = 0$ indicates rejection.

$\text{Simul}(\text{pp}, sk_V, pk_S, m) \xrightarrow{\S} \sigma'$: A PPT (or DPT) algorithm that, given the public parameters pp , the designated verifier's secret key sk_V , the signer's public key pk_S , and a message $m \in \mathcal{M}$, outputs a simulated designated verifier signature $\sigma' \in \Sigma$.

Correctness. Let pp be the public parameters generated by $\text{Setup}(1^\lambda)$, and let (sk_S, pk_S) and (sk_V, pk_V) be the key pairs generated by $\text{SigKeyGen}(\text{pp})$ and $\text{VerKeyGen}(\text{pp})$, respectively. The correctness of an SDVS scheme requires that, for any message $m \in \mathcal{M}$, the following conditions hold with overwhelming probability:

$$\text{Verify}(\text{pp}, sk_V, pk_S, m, \text{Sign}(\text{pp}, sk_S, pk_V, m)) = 1,$$

for a real designated verifier signature, and

$$\text{Verify}(\text{pp}, sk_V, pk_S, m, \text{Simul}(\text{pp}, sk_V, pk_S, m)) = 1,$$

for a simulated designated verifier signature. These conditions ensure that honestly generated real and simulated designated verifier signatures are both valid and accepted by the verifier.

2.2 Security Model

An SDVS scheme must ensure Strong Unforgeability under Chosen-Message Attacks (SUF-CMA), Non-Transferability (NT), and Privacy of Signer's Identity (PSI). The formal security definitions corresponding to these properties are presented below.

Strong Unforgeability under Chosen-Message Attacks (SUF-CMA) Security

In our SDVS scheme, CSI-SDVS, presented in Section 3, we adopt the notion of strong unforgeability, which is a stricter security requirement than existential unforgeability (EUF-CMA). This notion guarantees that no PPT adversary—without access to the secret key of either the signer or the designated verifier—can generate a valid designated verifier signature, even on messages that were previously queried via either the signing or simulating oracles. The formal definition is given below.

Definition 2.2 (SUF-CMA Security). *A strong designated verifier signature (SDVS) scheme is strongly unforgeable under chosen message attacks (SUF-CMA) if the advantage of any PPT adversary $\mathcal{A}$, defined as*

$$\text{Adv}_{\text{SDVS}, \mathcal{A}}^{\text{SUF-CMA}}(\lambda) = \Pr \left[\mathcal{A} \text{ wins } \text{Exp}_{\text{SDVS}, \mathcal{A}}^{\text{SUF-CMA}}(\lambda) \right],$$

is negligible. Here, $\text{Exp}_{\text{SDVS}, \mathcal{A}}^{\text{SUF-CMA}}(\lambda)$ denotes the SUF-CMA experiment between a challenger $\mathcal{C}$ and the adversary $\mathcal{A}$, as illustrated in Figure 1.

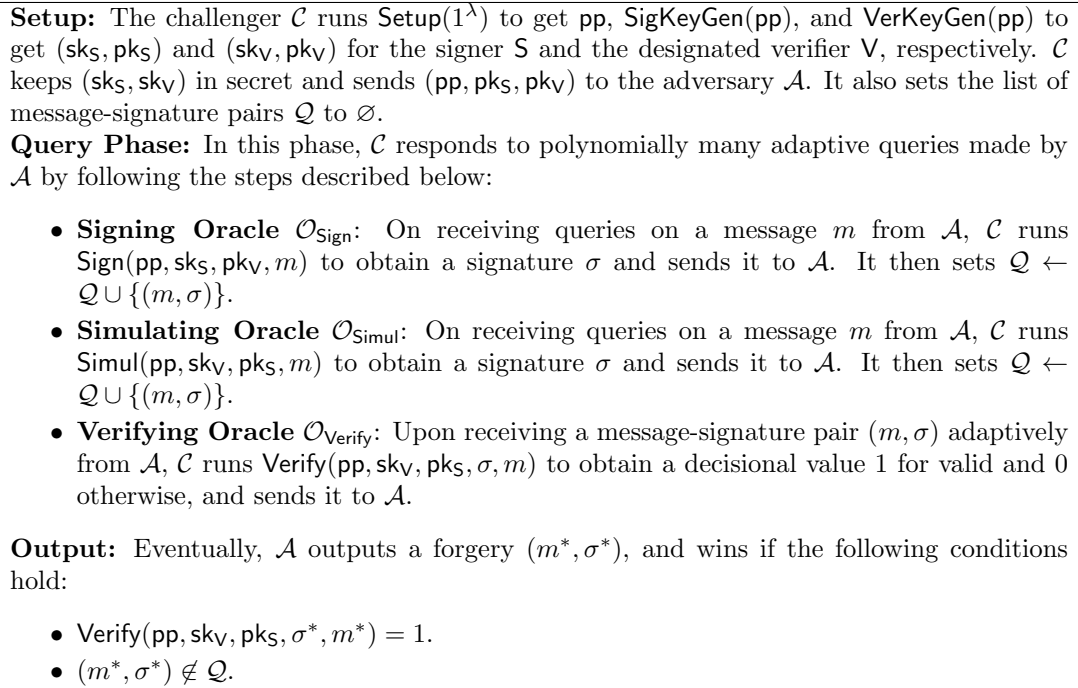


Figure 1: $\text{Exp}_{\text{SDVS}, \mathcal{A}}^{\text{SUF-CMA}}$ Experiment.

The probability is taken over the randomness of $\mathcal{A}$, and the algorithms **Sign** and **Simul**.

Non-Transferability (NT) Security

In an SDVS scheme, the notion of non-transferability (NT) ensures that the designated verifier cannot convince any third party of the authenticity of a valid signature. This is achieved by enabling the verifier to use the **Simul** algorithm to generate simulated signatures that are computationally indistinguishable from genuine ones produced by the signer. As a result, any conviction about the validity of a signature cannot be reliably transferred to others. The NT security of our SDVS construction is formalized as follows.

Definition 2.3 (NT Security). *A strong designated verifier signature (SDVS) scheme satisfies non-transferability (NT) security if the advantage of any PPT adversary $\mathcal{A}$, defined as*

$$\text{Adv}_{\text{SDVS}, \mathcal{A}}^{\text{NT}}(\lambda) = \Pr \left[\mathcal{A} \text{ wins } \text{Exp}_{\text{SDVS}, \mathcal{A}}^{\text{NT}}(\lambda) \right],$$

is negligible. Here, $\text{Exp}_{\text{SDVS}, \mathcal{A}}^{\text{NT}}(\lambda)$ denotes the NT experiment between a challenger $\mathcal{C}$ and the adversary $\mathcal{A}$, as illustrated in Figure 2.

Setup: The challenger $\mathcal{C}$ begins by executing **Setup**(1^λ) to generate the public parameters pp . It then generates key pairs for both the signer $\mathcal{S}$ and the designated verifier $\mathcal{V}$ by running **SigKeyGen**(pp) and **VerKeyGen**(pp), obtaining $(\text{sk}_\mathcal{S}, \text{pk}_\mathcal{S})$ and $(\text{sk}_\mathcal{V}, \text{pk}_\mathcal{V})$, respectively. The public keys $(\text{pk}_\mathcal{S}, \text{pk}_\mathcal{V})$ are then provided to the adversary $\mathcal{A}$.

Challenge: The adversary $\mathcal{A}$ adaptively selects a target message m^* . In response, $\mathcal{C}$ generates a real signature σ_0^* by running **Sign**($\text{pp}, \text{sk}_\mathcal{S}, \text{pk}_\mathcal{V}, m^*$), and also computes a simulated signature σ_1^* using **Simul**($\text{pp}, \text{sk}_\mathcal{V}, \text{pk}_\mathcal{S}, m^*$). $\mathcal{C}$ chooses a random bit $b \in \{0, 1\}$, and returns the signature σ_b^* to $\mathcal{A}$.

Output: Eventually, $\mathcal{A}$ outputs a bit b^* . The challenge is considered successful if $b^* = b$.

Figure 2: $\text{Exp}_{\text{SDVS}, \mathcal{A}}^{\text{NT}}$ Experiment.

The advantage of $\mathcal{A}$ in the above game is defined by

$$\text{Adv}_{\text{SDVS}, \mathcal{A}}^{\text{NT}}(\lambda) = \left| \Pr[b^* = b] - \frac{1}{2} \right|,$$

where the probability is taken over the randomness of $\mathcal{A}$, and the algorithms **Sign** and **Simul**.

Privacy of Signer's Identity (PSI) Security

The privacy of signer's identity (PSI) security notion ensures that the identity of the actual signer remains hidden. Within an SDVS scheme, even when a signature and two valid signing key pairs are available, no adversary observing the communication between the signer and the designated verifier can determine—except with negligible probability—which of the two secret keys was used to generate the signature. This anonymity is achieved by incorporating the designated verifier's secret key into the verification procedure, making it infeasible to link the signature to a specific signer.

Definition 2.4 (PSI Security). *A strong designated verifier signature (SDVS) scheme satisfies privacy of signer's identity (PSI) security if the advantage of any PPT adversary $\mathcal{A}$, defined as*

$$\text{Adv}_{\text{SDVS},\mathcal{A}}^{\text{PSI}}(\lambda) = \Pr \left[\mathcal{A} \text{ wins } \text{Exp}_{\text{SDVS},\mathcal{A}}^{\text{PSI}}(\lambda) \right],$$

is negligible. Here, $\text{Exp}_{\text{SDVS},\mathcal{A}}^{\text{PSI}}(\lambda)$ denotes the PSI experiment between a challenger $\mathcal{C}$ and the adversary $\mathcal{A}$, as illustrated in Figure 3.

Setup: The challenger $\mathcal{C}$ begins by executing $\text{Setup}(1^\lambda)$ to generate the public parameters pp . Next, it invokes $\text{SigKeyGen}(\text{pp})$ twice to obtain two signing key pairs: $(\text{sk}_{S_0}, \text{pk}_{S_0})$ and $(\text{sk}_{S_1}, \text{pk}_{S_1})$, corresponding to signers S_0 and S_1 , respectively. It also runs $\text{VerKeyGen}(\text{pp})$ to generate the verifier's key pair $(\text{sk}_V, \text{pk}_V)$. The challenger keeps sk_V private and sends the tuple $(\text{pp}, \text{sk}_{S_0}, \text{pk}_{S_0}, \text{sk}_{S_1}, \text{pk}_{S_1}, \text{pk}_V)$ to the adversary $\mathcal{A}$.

Query Phase: In this phase, $\mathcal{C}$ responds to polynomially many adaptive queries made by $\mathcal{A}$ by following the steps described below:

- **Simulating Oracle $\mathcal{O}_{\text{Simul}}$:** Upon receiving queries on message m and a bit b from $\mathcal{A}$, $\mathcal{C}$ computes a simulated signature σ_b by executing $\text{Simul}(\text{pp}, \text{sk}_V, \text{pk}_{S_b}, m)$ and returns it to $\mathcal{A}$.
- **Verifying Oracle: $\mathcal{O}_{\text{Verify}}$:** Upon receiving queries on message-signature pairs (m, σ) along with a bit $b \in \{0, 1\}$ from $\mathcal{A}$, $\mathcal{C}$ computes $\text{Verify}(\text{pp}, \text{sk}_V, \text{pk}_{S_b}, \sigma, m)$ and returns 1 if the signature is valid and 0 otherwise.

Challenge: The adversary $\mathcal{A}$ selects a challenge message m^* . Upon this, $\mathcal{C}$ computes two genuine signatures: σ_0^* by running $\text{Sign}(\text{pp}, \text{sk}_{S_0}, \text{pk}_V, m^*)$, and σ_1^* by running $\text{Sign}(\text{pp}, \text{sk}_{S_1}, \text{pk}_V, m^*)$. Then, a bit $b \in \{0, 1\}$ is chosen uniformly at random, and σ_b^* is sent to $\mathcal{A}$.

Output: Eventually, $\mathcal{A}$ outputs a guess $b^* \in \{0, 1\}$. It succeeds if both of the following conditions are met:

- $b^* = b$;
- The pair $(\sigma_{b^*}^*, m^*)$ was not previously submitted in a verification query for $b^* \in \{0, 1\}$.

Figure 3: $\text{Exp}_{\text{SDVS},\mathcal{A}}^{\text{PSI}}$ Experiment.

The advantage of $\mathcal{A}$ in the above game is defined by

$$\text{Adv}_{\text{SDVS},\mathcal{A}}^{\text{PSI}}(\lambda) = \left| \Pr[b^* = b] - \frac{1}{2} \right|,$$

where the probability is taken over the randomness of $\mathcal{A}$, and the algorithms Sign and Simul .

2.3 Elliptic Curves and Isogenies

This section recalls several fundamental properties of supersingular elliptic curves that are relevant to our construction. For a comprehensive treatment of the theory of elliptic curves, we refer the reader to [25, 26].

Let $k := \mathbb{F}_q$ denote a finite field, where $q := p^n$ for a prime $p > 3$ and a positive integer n . An *elliptic curve* E over k is a smooth, projective, genus-1 curve defined over k with a distinguished point $\mathcal{O}_E$ serving as the identity element for the group law on E . For a positive integer ℓ , the ℓ -*torsion subgroup* of E is defined as $E[\ell] := \{P \in E(\bar{k}) \mid [\ell]P = \mathcal{O}_E\}$. An elliptic curve is called *supersingular* if its group of p -torsion points is trivial over $\mathbb{F}_p$, i.e., $E[p] = \{\mathcal{O}_E\}$. In particular, if $E/\mathbb{F}_p$ is supersingular, then $\#E(\mathbb{F}_p) = p + 1$.

An *isogeny* is a surjective morphism between elliptic curves that also preserves the group structure. Two elliptic curves E_1 and E_2 defined over $\mathbb{F}_q$ are said to be *isogenous* if there exists an isogeny between them. For any finite subgroup $G \subset E(\mathbb{F}_q)$, there exists a unique (up to isomorphism) separable isogeny $\phi : E \rightarrow E' := E/G$ with kernel $\ker(\phi) = G$.

An *endomorphism* is an isogeny from an elliptic curve E to itself. Examples include the multiplication-by- m map $[m] : P \mapsto [m]P$ for $m \in \mathbb{Z}$, and the Frobenius endomorphism $\pi : (x, y) \mapsto (x^q, y^q)$ for an elliptic curve defined over $\mathbb{F}_q$. The set of all endomorphisms of E forms a ring under addition and composition, known as the *endomorphism ring* of E and denoted by $\text{End}(E)$. For a supersingular elliptic curve E defined over $\mathbb{F}_{p^2}$, the $\text{End}(E)$ is isomorphic to a maximal order in a quaternion algebra that is ramified at p and ∞ . In contrast, for a supersingular elliptic curve defined over $\mathbb{F}_p$, the ring of $\mathbb{F}_p$ -rational endomorphisms, denoted by $\text{End}_{\mathbb{F}_p}(E)$, is isomorphic to an order in some imaginary quadratic field. Hence, we have a strict inclusion $\text{End}_{\mathbb{F}_p}(E) \subset \text{End}(E)$.

Let $\mathfrak{D} \subset \mathbb{Q}(\sqrt{-p})$ be an order. The *ideal class group* $\text{Cl}(\mathfrak{D})$ is defined as the quotient $\mathcal{I}_{\mathfrak{D}}/\mathcal{P}_{\mathfrak{D}}$, where $\mathcal{I}_{\mathfrak{D}}$ denotes the group of invertible fractional ideals and $\mathcal{P}_{\mathfrak{D}}$ denotes the subgroup of principal fractional ideals. The class group $\text{Cl}(\mathfrak{D})$ acts naturally on the set $\mathcal{Ell}_p(\mathfrak{D})$ of $\mathbb{F}_p$ -isomorphism classes of supersingular elliptic curves defined over $\mathbb{F}_p$ with endomorphism ring isomorphic to a imaginary quadratic order $\mathfrak{D}$. For an $\mathfrak{D}$ -ideal $\mathfrak{a}$, the associated isogeny is determined by the kernel subgroup

$$S_{\mathfrak{a}} := \bigcap_{\alpha \in \mathfrak{a}} \ker(\alpha),$$

yielding the isogenous curve $\mathfrak{a} * E := E/S_{\mathfrak{a}}$. The isogeny $\phi_{\mathfrak{a}} : E \rightarrow E/S_{\mathfrak{a}}$ is well-defined and unique up to $\mathbb{F}_p$ -isomorphism. This gives rise to the group action

$$* : \text{Cl}(\mathfrak{D}) \times \mathcal{Ell}_p(\mathfrak{D}) \rightarrow \mathcal{Ell}_p(\mathfrak{D}),$$

which is both free and transitive.

Notation. Following [24], if $\text{Cl}(\mathfrak{D})$ is cyclic with generator $\mathfrak{g}$, then the map $\mathbb{Z}/N\mathbb{Z} \hookrightarrow \text{Cl}(\mathfrak{D})$ defined by $a \mapsto \mathfrak{g}^a$ gives a convenient representation of class group elements. Here, $N = \#\text{Cl}(\mathfrak{D})$, and any element $\mathfrak{a} \in \text{Cl}(\mathfrak{D})$ can be written as $\mathfrak{g}^a$ for some $a \in \mathbb{Z}/N\mathbb{Z}$. Using the shorthand $[a]$ for $\mathfrak{g}^a$ and $[a]E := \mathfrak{g}^a * E$, it follows that $[a][b]E = [a + b]E$.

2.4 Hardness Assumptions

We begin by recalling the basic framework of one-way group actions and the notion of *Hard Homogeneous Spaces (HHS)*, which form the foundation for isogeny-based hardness assumptions used in our construction.

Definition 2.5. *Given a group G with identity element e , and a set X , then a (left) group action $*$ of G on X is a function*

$$* : G \times X \rightarrow X,$$

satisfying the following axioms:

- (i) *Identity*: $e * x = x$ for all $x \in X$.
- (ii) *Compatibility*: $g * (h * x) = (gh) * x$ for all $g, h \in G$ and all $x \in X$.

To construct public-key cryptographic schemes based on group actions, Couveignes [27] introduced the concept of the HHS, defined as follows.

Hard Homogeneous Spaces (HHS). A *(principal) homogeneous space* for a group G is a non-empty finite set X on which G acts freely and transitively. That is, for every $x, y \in X$, there exists a unique $g \in G$ such that $y = g * x$. A *hard homogeneous space* (HHS) is a finite principal homogeneous space that satisfies the following two conditions:

- i. The following operations are *efficiently computable* (i.e., executable in polynomial time):
 - (Group Operations) Given $g, h \in G$, compute g^{-1} , gh , and decide if $g = h$.
 - (Random Element) Sample a random element in G with uniform probability.
 - (Membership) Given a string x_0 , decide if x_0 represents an element in X .
 - (Equality) Given $x_1, x_2 \in X$, decide if $x_1 = x_2$.
 - (Action) Given $g \in G$ and $x \in X$, then compute $g * x$.
- ii. The following computational problems are assumed to be *hard* (i.e., infeasible to solve in polynomial time):
 - *Vectorization*: Given $x, y \in X$, find $g \in G$ such that $y = g * x$.
 - *Parallelization*: Given $x, y, z \in X$ such that $y = g * x$, compute $z' = g * z$.

Our isogeny-based construction, presented in Section 3, is built upon the hardness of the *Group Action Inverse Problem (GAIP)*, which captures the difficulty of inverting group actions induced by ideal class groups on supersingular elliptic curves. This assumption underpins the security of the CSI-FiSh signature scheme [24].

Problem 2.6 (Group Action Inverse Problem (GAIP) [24]). *Given two supersingular elliptic curves E and E' over the same finite field $\mathbb{F}_p$ with $\text{End}_{\mathbb{F}_p}(E) \cong \text{End}_{\mathbb{F}_p}(E') \cong \mathfrak{O}$, the goal is to find an ideal $\mathfrak{a} \subset \mathfrak{O}$ such that $E' = \mathfrak{a} * E$.*

The security of CSI-FiSh—and by extension, the CSI-SDVS scheme introduced in Section 3—relies on a stronger assumption known as the *Multi-Target Group Action Inverse Problem (MT-GAIP)*. As demonstrated in [28], MT-GAIP is tightly reducible to GAIP when the structure of the class group is known, which is the case in the CSIDH-based setting.

Problem 2.7 (Multi-Target Group Action Inverse Problem (MT-GAIP) [24]). *Let $E_0, E_1, \dots, E_k$ be $k + 1$ supersingular elliptic curves defined over $\mathbb{F}_p$, with $\text{End}_{\mathbb{F}_p}(E_i) \cong \mathfrak{O}$ for all $0 \leq i \leq k$. Find an ideal $\mathfrak{a} \subset \mathfrak{O}$ such that $E_i = \mathfrak{a} * E_j$ for some distinct indices $i, j \in \{0, \dots, k\}$.*

The best known classical algorithm for solving GAIP (Problem 2.6) and its multi-target variant (Problem 2.7) runs in time $\mathcal{O}(\sqrt{N})$, where $N = \#\text{Cl}(\mathfrak{O})$. On the quantum side, Kuperberg's algorithm for the hidden shift problem [29, 30] achieves subexponential complexity. However, its concrete effectiveness in this setting remains an active topic of research [31, 32].

3 The Scheme: CSI-SDVS

In this section, we present CSI-SDVS, a new isogeny-based SDVS scheme. The construction is based on the CSIDH [23] framework and adopts the known ideal class group structure introduced in the CSI-FiSh signature scheme [24]. Specifically, CSI-FiSh exploits the precomputed structure of the ideal class group $\text{Cl}(\mathfrak{D})$ for the CSIDH-512 parameter set, represented as a relation lattice of low-norm generators. In this setting, $\text{Cl}(\mathfrak{D})$ is cyclic of order N , allowing each element to be uniquely expressed as $[\mathfrak{g}^a]$, where $\mathfrak{g}$ is a fixed generator and $a \in \mathbb{Z}_N$. By leveraging this explicit cyclic structure, both deterministic computation and uniform sampling—achieved by exponentiating the fixed generator $\mathfrak{g}$ in $\mathbb{Z}_N$ —are naturally supported, preserving the post-quantum security guarantees provided by the class group action framework. Consequently, CSI-SDVS achieves efficient and verifiable designated verifier signatures without relying on heuristic assumptions.

In CSI-SDVS, the key generation procedures for both the signer and the designated verifier closely follow the key generation phase of CSI-FiSh. The core of the **Sign** procedure—also mirrored in **Simul**—is a structured interaction between class group elements and $\mathbb{F}_p$ -isomorphism classes of supersingular elliptic curves. The signer begins with an intermediate curve $\hat{E} = [v]E_0$, samples a secret $b \in \mathbb{Z}_N$, and computes the target curve $Y = [b]\hat{E}$ via the class group action. The designated verifier, using its secret key $v \in \mathbb{Z}_N$, then computes $Y = [v]\bar{E} = [v + b]E_0$ from the intermediate curve $\bar{E} = [b]E_0$. This resulting curve constitutes the central component for verifying the designated verifier signature.

The complete scheme consists of six polynomial-time algorithms, described in detail in the following procedures.

CSI-SDVS.Setup(1^λ) $\rightarrow$ pp: A trusted party executes the setup procedure to generate the public parameters **pp** as follows. First, it selects a large prime $p = 4\ell_1\ell_2 \dots \ell_n - 1$, where the ℓ_i are small, distinct, odd primes. Specifically, it sets $n = 74$, $\ell_1 = 3$, $\ell_{73} = 373$, and $\ell_{74} = 587$. Next, it fixes a base curve $E_0 : y^2 = x^3 + x$ defined over $\mathbb{F}_p$, such that $E_0 \in \mathcal{Ell}_p(\mathfrak{D})$, and chooses a generator $\mathfrak{g}$ of the class group $\mathcal{G} = \text{Cl}(\mathfrak{D})$ with class number $N \approx p^{1/2}$, where $\mathfrak{D} = \mathbb{Z}[\sqrt{-p}]$. It then samples a cryptographically secure hash function $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, where λ is a security parameter. In addition, a positive integer η is specified. Finally, it outputs the public parameters $\text{pp} := (p, \mathfrak{g}, N, E_0, \mathcal{H}, \eta)$.

CSI-SDVS.SigKeyGen(pp) $\rightarrow$ (sk_S, pk_S): Given **pp**, the signer generates a key pair as follows:

1. Sample $s_i \in \mathbb{Z}_N$ uniformly at random and compute $E_i := [s_i]E_0$ for each $i \in [\eta]$.
2. Set the signer's secret/public key pair as $(\text{sk}_S, \text{pk}_S) := (\{s_i\}_{i=1}^\eta, \{E_i\}_{i=1}^\eta)$.

CSI-SDVS.VerKeyGen(pp) $\rightarrow$ (sk_V, pk_V): Given **pp**, the designated verifier generates a key pair as follows:

1. Sample $v_i \in \mathbb{Z}_N$ uniformly at random and compute $\hat{E}_i := [v_i]E_0$ for each $i \in [\eta]$.
2. Set the verifier's secret/public key pair as $(\text{sk}_V, \text{pk}_V) := (\{v_i\}_{i=1}^\eta, \{\hat{E}_i\}_{i=1}^\eta)$.

CSI-SDVS.Sign(pp, sk_S, pk_V, m) $\rightarrow$ σ : Employing **pp**, the secret key $\text{sk}_S = \{s_i\}_{i=1}^\eta$, and the designated verifier's public key $\text{pk}_V = \{\hat{E}_i\}_{i=1}^\eta$, a signer computes the designated verifier signature σ on $m \in \{0, 1\}^*$ as follows:

1. Sample $b_i \in \mathbb{Z}_N$ uniformly at random for each $i \in [\eta]$.

2. Compute the curves $Y_i := [b_i]\hat{E}_i$ for each $i \in [\eta]$.
3. Compute the hash value $h := \mathcal{H}(Y_1 \parallel \dots \parallel Y_\eta \parallel m)$.
4. Compute the vector $z_i := b_i - s_i \pmod N$ for each $i \in [\eta]$.
5. Output the designated verifier signature $\sigma := (h, \mathbf{z})$, where $\mathbf{z} := \{z_i\}_{i=1}^\eta$.

CSI-SDVS.Verify($\text{pp}, \text{sk}_V, \text{pk}_S, m, \sigma$) $\rightarrow \{0, 1\}$: Incorporating pp , the secret key $\text{sk}_V = \{v_i\}_{i=1}^\eta$, and the signer's public key $\text{pk}_S = \{E_i\}_{i=1}^\eta$, a designated verifier checks the validity of the designated verifier signature σ on $m \in \{0, 1\}^*$ as follows:

1. Parse $\sigma := (h, \mathbf{z})$, where $\mathbf{z} := \{z_i\}_{i=1}^\eta$.
2. Compute $y_i := v_i + z_i \pmod N$ for each $i \in [\eta]$.
3. Compute the curves $Y'_i := [y_i]E_i$ for each $i \in [\eta]$.
4. Recover the hash value $h' := \mathcal{H}(Y'_1 \parallel \dots \parallel Y'_\eta \parallel m)$.
5. Output 1 if $h' = h$, otherwise 0.

CSI-SDVS.Simul($\text{pp}, \text{sk}_V, \text{pk}_S, m$) $\rightarrow \sigma$: Using pp , the secret key $\text{sk}_V = \{v_i\}_{i=1}^\eta$, and the signer's public key $\text{pk}_S = \{E_i\}_{i=1}^\eta$, a designated verifier computes the simulated designated verifier signature σ on $m \in \{0, 1\}^*$ as follows:

1. Sample $r_i \in \mathbb{Z}_N$ uniformly at random for each $i \in [\eta]$.
2. Compute the curves $Y_i := [r_i]E_i$ for each $i \in [\eta]$.
3. Compute the hash value $h := \mathcal{H}(Y_1 \parallel \dots \parallel Y_\eta \parallel m)$.
4. Compute the vector $z_i := r_i - v_i \pmod N$ for each $i \in [\eta]$.
5. Output the simulated designated verifier signature $\sigma := (h, \mathbf{z})$, where $\mathbf{z} := \{z_i\}_{i=1}^\eta$.

Correctness. The proposed scheme satisfies the correctness requirement stated in Definition 2.1. In particular, during the verification phase, the designated verifier validates a message-signature pair (m, σ) using the inputs $(\text{pp}, \text{sk}_V, \text{pk}_S)$, where $\text{sk}_V = \{v_i\}_{i=1}^\eta$ and $\text{pk}_S = \{E_i\}_{i=1}^\eta$. We distinguish two correctness cases corresponding to the two ways a designated verifier signature may be generated, and verify that in both cases the verification algorithm accepts.

- i.* When the designated verifier signature is generated by the actual signer: Let $\sigma := (h, \mathbf{z})$, where $\mathbf{z} := \{z_i\}_{i=1}^\eta$. The verifier computes each $Y_i = [b_i]\hat{E}_i$ as follows:

$$\begin{aligned}
 Y_i &= [v_i][z_i]E_i \\
 &= [v_i + b_i - s_i][s_i]E_0 \\
 &= [v_i + b_i]E_0 \\
 &= [b_i]\hat{E}_i,
 \end{aligned}$$

which implies that $h = \mathcal{H}(Y_1 \parallel \dots \parallel Y_\eta \parallel m)$, as expected.

- ii. When the signature is simulated by the designated verifier:* Let $\sigma' := (h, \mathbf{z})$, where $\mathbf{z} := \{z_i\}_{i=1}^\eta$ is computed using the Simul algorithm. Then each value $Y_i = [r_i]E_i$ is recovered by computing:

$$\begin{aligned} Y_i &= [v_i][z_i]E_i \\ &= [v_i + r_i - v_i]E_i \\ &= [r_i]E_i, \end{aligned}$$

which again yields $h = \mathcal{H}(Y_1 \| \dots \| Y_\eta \| m)$.

4 Efficiency Analysis

In this section, we analyze the communication and storage overhead of CSI-SDVS. The parameter $\eta \geq 1$ represents the number of parallel instances in the base construction. While $\eta = 1$ is considered sufficient for current parameter choices, keeping η explicit provides several advantages. These include flexibility for potential future instantiations with $\eta > 1$, stronger resistance against multi-target attacks due to increased hash input entropy, and a unified framework for security analysis.

For a security level of $\lambda = 128$, we have the following:

- The secret keys $\mathbf{sk}_S = \{s_i \bmod N\}_{i=1}^\eta$ and $\mathbf{sk}_V = \{v_i \bmod N\}_{i=1}^\eta$, held by the signer and designated verifier respectively, consist of η elements in $\mathbb{Z}_N$. Since $\log_2 N = 2\lambda$, then the total size of each secret key is $\eta \log_2 N = \mathcal{O}(\lambda)$ bits.
- Each curve in $\mathcal{Ell}_p(\mathfrak{D})$ is uniquely represented by a Montgomery coefficient $A \in \mathbb{F}_p$, defining a curve $E_A : y^2 = x^3 + Ax^2 + x$. Consequently, the public keys $\mathbf{pk}_S$ and $\mathbf{pk}_V$ each contains η such elements over $\mathbb{F}_p$, yielding a size of $\eta \log_2 p = \mathcal{O}(\lambda)$ bits.
- A signature $\sigma = (h, \mathbf{z})$, where $\mathbf{z} = \{z_i \bmod N\}_{i=1}^\eta$, consists of a λ -bit hash together with η elements in $\mathbb{Z}_N$, each requiring at most 2λ bits. Hence, the total signature size is $(2\eta + 1)\lambda = \mathcal{O}(\lambda)$ bits.

5 Security Analysis of CSI-SDVS

Theorem 5.1. *The CSI-SDVS scheme introduced in Section 3 is strongly unforgeable under chosen message attacks (SUF-CMA), as defined in Definition 2.2.*

Proof. We prove that the CSI-SDVS scheme satisfies SUF-CMA security under Definition 2.2. Suppose, for the sake of contradiction, that there exists a PPT adversary $\mathcal{A}$ that succeeds in the experiment $\text{Exp}_{\text{CSI-SDVS}, \mathcal{A}}^{\text{SUF-CMA}}(\lambda)$ (illustrated in Figure 1) with non-negligible probability. Strictly speaking, the simulator $\mathcal{S}$ first interacts with its challenger $\mathcal{C}$ to obtain the public keys of the signer and the designated verifier, and subsequently simulates all oracle responses for $\mathcal{A}$ without ever accessing the corresponding secret keys. For brevity and without loss of generality, we present the proof as if $\mathcal{S}$ itself plays the role of the challenger, directly answering $\mathcal{A}$'s queries via the signing oracle $\mathcal{O}_{\text{Sign}}$, the simulating oracle $\mathcal{O}_{\text{Simul}}$, the verification oracle $\mathcal{O}_{\text{Verify}}$, and the random oracle $\mathcal{O}_{\mathcal{H}}$.

At the end of the interaction, $\mathcal{A}$ outputs a forged signature $\sigma^* := (h^*, \mathbf{z}^*)$ on a message m^* with non-negligible probability, satisfying

$$\forall i \in [\eta] \quad Y_i^* = [v_i + z_i^*]E_i, \quad \text{and} \quad h^* = \mathcal{H}(Y_1^* \| \dots \| Y_\eta^* \| m^*).$$

Leveraging the forgery σ^* , the simulator $\mathcal{S}$ computes the secret curves $[s_i + v_i]E_0$ from the public values $[s_i]E_0$ and $[v_i]E_0$, thereby breaking the assumed hardness of the parallelization problem in the HHS model, as defined in Section 2.4.

Setup. Given public parameters $\text{pp} := (p, \mathbf{g}, N, E_0, \mathcal{H}, \eta)$, as defined in Section 3, the simulator $\mathcal{S}$ proceeds as follows.

1. For each $i \in [\eta]$, sample $s_i \in \mathbb{Z}_N$ uniformly at random and compute $E_i := [s_i]E_0$. Set the signer's key pair as $(\text{sk}_S, \text{pk}_S) := (\{s_i\}_{i=1}^\eta, \{E_i\}_{i=1}^\eta)$.
2. For each $i \in [\eta]$, sample $v_i \in \mathbb{Z}_N$ uniformly at random and compute $\hat{E}_i := [v_i]E_0$. Set the verifier's key pair as $(\text{sk}_V, \text{pk}_V) := (\{v_i\}_{i=1}^\eta, \{\hat{E}_i\}_{i=1}^\eta)$.
3. Send $(\text{pp}, \text{pk}_S, \text{pk}_V)$ to the adversary $\mathcal{A}$, while keeping $(\text{sk}_S, \text{sk}_V) = (\{s_i\}_{i=1}^\eta, \{v_i\}_{i=1}^\eta)$ secret, and define sets $\mathcal{Q} := \emptyset$, $\text{HList} := \emptyset$.

Simulation of Queries. $\mathcal{A}$ issues polynomially many adaptive queries to the oracles $\mathcal{O}_{\mathcal{H}}$, $\mathcal{O}_{\text{Sign}}$, $\mathcal{O}_{\text{Simul}}$, and $\mathcal{O}_{\text{Verify}}$.

- *Hashing Oracle $\mathcal{O}_{\mathcal{H}}$:* Upon receiving a query on a tuple $(m, \{Y_i\}_{i=1}^\eta)$, the simulator $\mathcal{S}$ checks whether a corresponding hash output h is already stored in the hash list HList . If such an entry exists, $\mathcal{S}$ returns h directly. Otherwise, it samples $h \xleftarrow{\$} \{0, 1\}^\lambda$ at random, programs the oracle as $h =: \mathcal{H}(Y_1 \parallel \dots \parallel Y_\eta \parallel m)$ and returns h to $\mathcal{A}$. Finally, $\mathcal{S}$ updates the hash list as $\text{HList} \leftarrow \text{HList} \cup \{(m, \{Y_i\}_{i=1}^\eta, h, \perp)\}$.
- *Signing Oracle $\mathcal{O}_{\text{Sign}}$:* In response to a query on a message m , $\mathcal{S}$ produces a signature as follows.
 - Sample $z_i \in \mathbb{Z}_N$, and compute the curves $\bar{E}_i := [z_i]E_i$ for each $i \in [\eta]$.
 - Sample $v_i \in \mathbb{Z}_N$, and compute the curves $Y_i := [v_i]\bar{E}_i$ for each $i \in [\eta]$.
 - Sample $h \xleftarrow{\$} \{0, 1\}^\lambda$ at random, program the oracle as $h =: \mathcal{H}(Y_1 \parallel \dots \parallel Y_\eta \parallel m)$, and update the hash list as $\text{HList} \leftarrow \text{HList} \cup \{(m, \{Y_i\}_{i=1}^\eta, h, \{\bar{E}_i\}_{i=1}^\eta)\}$.
 - Return $\sigma := (h, \mathbf{z})$ to $\mathcal{A}$, where $\mathbf{z} := \{z_i\}_{i=1}^\eta$. It updates the set $\mathcal{Q}$ as $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(m, \sigma)\}$.
- *Simulating Oracle $\mathcal{O}_{\text{Simul}}$:* In response to a query on a message m , the simulator $\mathcal{S}$ produces a simulated designated verifier signature with a similar arguments as in the Signing Oracle $\mathcal{O}_{\text{Sign}}$, updates the set $\mathcal{Q}$ as $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(m, \sigma)\}$, and returns a signature $\sigma := (h, \mathbf{z})$ to $\mathcal{A}$.
- *Verifying oracle $\mathcal{O}_{\text{Verify}}$:* Upon receiving a query on message-signature pair (m, σ) to this oracle, $\mathcal{S}$ proceeds as follows.
 - Parse $\sigma := (h, \mathbf{z})$, where $\mathbf{z} := \{z_i\}_{i=1}^\eta$.
 - Check whether there exist an entry $(m, \{Y_i\}_{i=1}^\eta, h, \{\bar{E}_i\}_{i=1}^\eta) \in \text{HList}$ such that $m = m$ and $h = h$.
 - Extract $\{\bar{E}_i\}_{i=1}^\eta$ from $(m, \{Y_i\}_{i=1}^\eta, h, \{\bar{E}_i\}_{i=1}^\eta)$, and check if $[z_i]E_i = \bar{E}_i$ or for all $i \in [\eta]$.
 - Output 1 if all the above are satisfied, otherwise 0.

Extracting the Forgery. Eventually, the adversary $\mathcal{A}$ outputs a forgery (m^*, σ^*) with non-negligible probability, where $\sigma^* := (h^*, \mathbf{z}^*)$. The simulator $\mathcal{S}$ then searches for a matching entry $(m, h, \{Y_i\}_{i=1}^\eta, \perp)$ in the hash list **HList** such that $m^* = m$, $h^* = h$, and the last component is $\perp$. If such an entry is found, $\mathcal{S}$ can solve an instance of the parallelization problem in the HHS model, as given in Section 2.4, with a non-negligible probability by recovering the secret value $[s_i + v_i]E_0$ as follows:

$$[-z_i^*]Y_i = [s_i - b_i^*][b_i^*]\hat{E}_i = [s_i]\hat{E}_i = [s_i + v_i]E_0, \quad \text{for each } i \in [\eta],$$

which is induced by the secret keys $\{s_i\}_{i=1}^\eta$ and $\{v_i\}_{i=1}^\eta$ of the signer and designated verifier, respectively. $\square$

Theorem 5.2. *The CSI-SDVS scheme presented in Section 3 satisfies non-transferability (NT) security, as per Definition 2.3.*

Proof. According to the NT security model of the SDVS scheme defined in Definition 2.3, a game is established between the PPT distinguisher $\mathcal{A}$ and the simulator $\mathcal{B}$ as follows.

Setup: $\mathcal{B}$ performs the following operations:

1. Execute $\text{Setup}(1^\lambda)$ to obtain the public parameters pp .
2. Run $\text{SigKeyGen}(\text{pp})$ to generate the signer's key pair $(\text{sk}_S, \text{pk}_S) = (\{s_i\}_{i=1}^\eta, \{E_i\}_{i=1}^\eta)$.
3. Run $\text{VerKeyGen}(\text{pp})$ to generate the designated verifier's key pair $(\text{sk}_V, \text{pk}_V) = (\{v_i\}_{i=1}^\eta, \{\hat{E}_i\}_{i=1}^\eta)$.
4. Provide $\mathcal{A}$ with $(\text{pp}, \text{pk}_S, \text{pk}_V)$, while retaining sk_S and sk_V as private.

Challenge: Upon receiving a message m from $\mathcal{A}$, $\mathcal{B}$ proceeds as follows:

1. Invoke $\text{Sign}(\text{pp}, \text{sk}_S, \text{pk}_V, m)$ to obtain a real designated verifier signature $\sigma_0 = (h^{(0)}, \mathbf{z}^{(0)})$.
2. Invoke $\text{Simul}(\text{pp}, \text{sk}_V, \text{pk}_S, m)$ to obtain a simulated designated verifier signature $\sigma_1 = (h^{(1)}, \mathbf{z}^{(1)})$.
3. Select a random bit $b \xleftarrow{\$} \{0, 1\}$ and send the pair (m, σ_b) to $\mathcal{A}$.

Output: $\mathcal{A}$ responds with a guess $b^* \in \{0, 1\}$.

In the above process, the randomness of the vectors $h^{(0)}$ and $h^{(1)}$ within the range $\{0, 1\}^\lambda$ is ensured by the property of the hash function $\mathcal{H}$. For each $i = 1, \dots, \eta$, since the values $b_i^{(0)}$ and $b_i^{(1)}$ are sampled uniformly at random from $\mathbb{Z}_N$, then:

- The components

$$z_i^{(0)} = b_i^{(0)} - s_i \pmod{N} \quad \text{and} \quad z_i^{(1)} = b_i^{(1)} - v_i \pmod{N}$$

retain their uniform randomness. Consequently, the vectors $\mathbf{z}^{(0)} = (z_1^{(0)}, \dots, z_\eta^{(0)})$ and $\mathbf{z}^{(1)} = (z_1^{(1)}, \dots, z_\eta^{(1)})$ are both uniformly distributed over $\mathbb{Z}_N^\eta$ and are thus identically distributed.

- For the curves

$$[z_i^{(0)}]E_i = [b_i^{(0)} - s_i]E_i = [b_i^{(0)}]E_0 \quad \text{and} \quad [z_i^{(1)}]E_i = [b_i^{(1)} - v_i]E_i = [b_i^{(1)} - v_i + s_i]E_0,$$

the components

$$b_i^{(0)} \pmod{N} \quad \text{and} \quad b_i^{(1)} - v_i + s_i \pmod{N}$$

are uniformly distributed over $\mathbb{Z}_N$. Therefore, $[b_i^{(0)}]E_0$ and $[b_i^{(1)} - v_i + s_i]E_0$ are both uniformly distributed over $\mathcal{Ell}_p(\mathfrak{D})$.

As a result, the adversary $\mathcal{A}$ cannot distinguish whether the provided signature σ_b was generated by the signer or simulated. Hence, its advantage in guessing b correctly is negligible, thereby satisfying non-transferability as defined in Definition 2.3. $\square$

Theorem 5.3. *If the MT-GAIP defined in Section 2.4 is hard, then the CSI-SDVS scheme introduced in Section 3 satisfies privacy of signer's identity (PSI) security as defined in Definition 2.4.*

Proof. As specified in the PSI security model of the SDVS scheme, as per Definition 2.4, the interaction between the PPT adversary $\mathcal{A}$ and the challenger $\mathcal{C}$ is modeled as the following game.

Setup: The challenger $\mathcal{C}$ proceeds with the following steps:

1. Execute $\text{Setup}(1^\lambda)$ to generate the public parameters pp .
2. Run $\text{SigKeyGen}(\text{pp})$ twice to obtain key pairs for signers S_0 and S_1 as follows:

$$(\text{sk}_{S_0}, \text{pk}_{S_0}) = (\{s_i^{(0)}\}_{i=1}^\eta, \{E_i^{(0)}\}_{i=1}^\eta), \quad (\text{sk}_{S_1}, \text{pk}_{S_1}) = (\{s_i^{(1)}\}_{i=1}^\eta, \{E_i^{(1)}\}_{i=1}^\eta).$$

3. Run $\text{VerKeyGen}(\text{pp})$ to generate the designated verifier's key pair $(\text{sk}_V, \text{pk}_V) = (\{v_i\}_{i=1}^\eta, \{\hat{E}_i\}_{i=1}^\eta)$.
4. Send $(\text{pp}, \text{sk}_{S_0}, \text{pk}_{S_0}, \text{sk}_{S_1}, \text{pk}_{S_1}, \text{pk}_V)$ to $\mathcal{A}$, while keeping sk_V secret.

Query Phase: During this phase, $\mathcal{C}$ responds to polynomially many adaptive queries from $\mathcal{A}$ as follows:

- *Simulation Oracle $\mathcal{O}_{\text{Simul}}$:* Upon receiving queries on message m and a bit $b \in \{0, 1\}$ from $\mathcal{A}$, $\mathcal{C}$ computes a simulated designated verifier signature by running: $\text{Simul}(\text{pp}, \text{sk}_V, \text{pk}_{S_b}, m)$ and returns $\sigma_b = (h^{(b)}, \mathbf{z}^{(b)})$ to $\mathcal{A}$.
- *Verification Oracle $\mathcal{O}_{\text{Verify}}$:* Upon receiving queries on message-signature pair (m, σ) and bit $b \in \{0, 1\}$ from $\mathcal{A}$, $\mathcal{C}$ runs: $\text{Verify}(\text{pp}, \text{sk}_V, \text{pk}_{S_b}, \sigma, m)$ and returns 1 if the signature is valid and 0 otherwise.

Challenge: Upon receiving a challenge message m from $\mathcal{A}$, $\mathcal{C}$ executes the following:

1. Compute a real designated verifier signature σ_0 from S_0 by running:

$$\sigma_0 = (h^{(0)}, \mathbf{z}^{(0)}) \xleftarrow{\$} \text{Sign}(\text{pp}, \text{sk}_{S_0}, \text{pk}_V, m).$$

2. Similarly, compute a real designated verifier signature σ_1 from S_1 by running:

$$\sigma_1 = (h^{(1)}, \mathbf{z}^{(1)}) \xleftarrow{\$} \text{Sign}(\text{pp}, \text{sk}_{S_1}, \text{pk}_V, m).$$

3. Choose a random bit $b \xleftarrow{\$} \{0, 1\}$ and send (m, σ_b) to $\mathcal{A}$.

Output: Finally, $\mathcal{A}$ returns a guess bit $b^* \in \{0, 1\}$.

Similar to the reasoning used in the proof of NT security in Theorem 5.2, the randomness of $h^{(0)}$ and $h^{(1)}$ is ensured by the properties of the hash function $\mathcal{H}$, with each vector uniformly distributed over $\{0, 1\}^\lambda$. Likewise, the vectors $\mathbf{z}^{(0)} = \{z_i^{(0)}\}_{i=1}^\eta$ and $\mathbf{z}^{(1)} = \{z_i^{(1)}\}_{i=1}^\eta$ follow the same distribution as elements from $\mathbb{Z}_N^\eta$. Given a real message-signature pair (m, σ_b) , the adversary $\mathcal{A}$ is expected to proceed as follows:

1. Parse $\sigma_b = (h^{(b)}, \mathbf{z}^{(b)}) = (h_i^{(b)}, \{z_i^{(b)}\}_{i=1}^\eta)$.
2. Compute $X_i^{(j)} := [z_i^{(b)}]E_i^{(j)}$ for each $i \in [\eta]$ and $j = 0, 1$.
3. Compute $Y_i^{(j)} := [v_i]X_i^{(j)}$ for each $i \in [\eta]$ and $j = 0, 1$.
4. Check if $h^{(b)} = \mathcal{H}(Y_1^{(j)} \parallel \dots \parallel Y_\eta^{(j)} \parallel m)$ for $j = 0, 1$.

According to the definition of MT-GAIP in Problem 2.7, it is evident that, without access to the designated verifier's secret key $\mathbf{sk}_V = \{v_i\}_{i=1}^\eta$, even when $j = b$, the adversary $\mathcal{A}$ cannot compute the following set of curves:

$$Y_i^{(j)} := [v_i]X_i^{(j)}, \quad \text{for all } i \in [\eta].$$

in Step 3. As a result, the adversary cannot determine which signer produced the signature, as doing so would require recovering the curves $Y_i^{(b)}$ in order to compute $[s_i^{(b)} + v_i]E_0$:

$$\begin{aligned} [-z_i^{(b)}][Y_i^{(b)}] &= [s_i^{(b)} - b_i^{(b)}][b_i^{(b)}]\hat{E}_i \\ &= [s_i^{(b)}]\hat{E}_i \\ &= [s_i^{(b)} + v_i]E_0, \end{aligned}$$

which would enable distinguishing which of the two potential signers generated the signature σ_b . This guarantees that the signer's anonymity is preserved, and that adversary's distinguishing advantage remains negligible in the absence of $\mathbf{sk}_V = \{v_i\}_{i=1}^\eta$. $\square$

Conclusion

In this work, we introduced CSI-SDVS, a new post-quantum Strong Designated Verifier Signature (SDVS) scheme based on isogenies. By leveraging the class group action on supersingular elliptic curves and the hardness of the Multi-Target Group Action Inverse Problem (MT-GAIP), our construction achieves Strong Unforgeability (SUF-CMA), Non-Transferability (NT), and Privacy of the Signer's Identity (PSI) in the random oracle model. Compared to lattice-based SDVS schemes [14, 15, 16, 17], which typically require larger key and signature sizes due to high-dimensional trapdoor structures, CSI-SDVS achieves compact key and signature sizes, making it practical for resource-constrained environments. Although an earlier isogeny-based SDVS scheme [18] was proposed, it has been rendered insecure due to devastating cryptanalytic attacks [20, 21, 22] on SIDH. To the best of our knowledge, CSI-SDVS is the only isogeny-based SDVS scheme that achieves security against known quantum and classical attacks, along with compactness and efficiency.

References

- [1] Jakobsson, M., Sako, K., Impagliazzo, R.: Designated verifier proofs and their applications. *International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 143–154. Springer (1996)
- [2] Chaum, D.: Private signature and proof systems. In: *United States Patent*, 1996, <https://patents.google.com/patent/US5493614A/en>. (1996)
- [3] Laguillaumie, F., Vergnaud, D.: Designated verifier signatures: Anonymity and efficient construction from any bilinear map. *International Conference on Security in Communication Networks*, pp. 105–119. Springer (2004)
- [4] Saeednia, S., Kremer, S., Markowitch, O.: An efficient strong designated verifier signature scheme. *International conference on information security and cryptology* (pp. 40-54). Berlin, Heidelberg. Springer (2003)
- [5] Huang, X., Susilo, W., Mu, Y., Zhang, F.: Short (identity-based) strong designated verifier signature schemes. *International Conference on Information Security Practice and Experience* (pp. 214-225). Berlin, Heidelberg. Springer (2006)
- [6] Kang, B., Boyd, C., Dawson, E. D.: A novel identity-based strong designated verifier signature scheme. *Journal of Systems and Software*, 82(2), 270-273. (2009)
- [7] Shor, P. W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review*, pp. 303–332. SIAM (1999)
- [8] Asaar, M. R.: Code-based strong designated verifier signatures: security analysis and a new construction. *Cryptology ePrint Archive*. (2016)
- [9] Ren, Y., Wang, H., Du, J., Ma, L.: Code-based authentication with designated verifier. *International Journal of Grid and Utility Computing*, 7(1), pp.61-67. (2016)
- [10] Shooshtari, M. K., Ahmadian-Attari, M., Aref, M. R.: Provably secure strong designated verifier signature scheme based on coding theory. *International Journal of Communication Systems*, 30(7), p.e3162. (2017)
- [11] Process, P. S.: Third Round Candidate Announcement. *Information Technology Laboratory-Computer Security Resource Center*. (2020)
- [12] Thanalakshmi, P., Anitha, R.: A new code-based designated verifier signature scheme. *International Journal of Communication Systems*, 31(17), p.e3803. (2018)
- [13] Gentry, C., Peikert, C., Vaikuntanathan, V.: Trapdoors for hard lattices and new cryptographic constructions. *Proceedings of the fortieth annual ACM symposium on Theory of computing*, pp. 197–206. (2008)
- [14] Wang, F., Hu, Y., Wang, B.: Lattice-based strong designate verifier signature and its applications. *Malaysian Journal of Computer Science*, pp. 11–22. (2012)
- [15] Noh, G., Jeong, I. R.: Strong designated verifier signature scheme from lattices in the standard model. *Security and Communication Networks*, pp. 6202–6214. Wiley Online Library (2016)

- [16] Cai, J., Jiang, H., Zhang, P., Zheng, Z., Lyu, G., Xu, Q.: An efficient strong designated verifier signature based on $\mathcal{R}$ -SIS assumption. *IEEE Access*, pp. 3938–3947. IEEE (2019)
- [17] Zhang, Y., Susilo, W., Guo, F.: Lattice-based strong designated verifier signature with non-delegatability. *Computer Standards & Interfaces*, pp. 103904. Elsevier (2025)
- [18] Sun, X., Tian, H., Wang, Y.: Toward quantum-resistant strong designated verifier signature from isogenies. *Fourth International Conference on Intelligent Networking and Collaborative Systems*, pp. 292–296. IEEE (2012)
- [19] De Feo, L., Jao, D., Plût, J.: Towards quantum resistant cryptosystems from supersingular elliptic curve isogenies. *Cryptology ePrint Archive*, Paper 2011/506. <https://eprint.iacr.org/2011/506> (2011)
- [20] Castryck, W., Decru, T.: An efficient key recovery attack on SIDH. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 423–447. Springer (2023)
- [21] Maino, L., Martindale, C., Panny, L., Pope, G., Wesolowski, B.: A direct key recovery attack on SIDH. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 448–471. Springer (2023)
- [22] Robert, D.: Breaking SIDH in polynomial time. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 472–503. Springer (2023)
- [23] Castryck, W., Lange, T., Martindale, C., Panny, L., Renes, J.: CSIDH: an efficient post-quantum commutative group action. *Advances in Cryptology—ASIACRYPT 2018*, pp. 395–427. Springer (2018)
- [24] Beullens, W., Kleinjung, T., Vercauteren, F.: CSI-FiSh: efficient isogeny based signatures through class group computations. *International conference on the theory and application of cryptology and information security*, pp. 227–247. Springer (2019)
- [25] Washington, L. C.: *Elliptic curves: number theory and cryptography*. Chapman and Hall/CRC (2008)
- [26] Silverman, J. H.: *The arithmetic of elliptic curves*. Springer (2009)
- [27] Couveignes, J. M.: Hard Homogeneous Spaces. *Cryptology ePrint Archive*, Paper 2006/291 (2006)
- [28] De Feo, L., Galbraith, S. D.: SeaSign: compact isogeny signatures from class group actions. *EUROCRYPT 2019*, pp. 759–789. Springer (2019)
- [29] Kuperberg, G.: A subexponential-time quantum algorithm for the dihedral hidden subgroup problem. *SIAM Journal on Computing*, pp. 170–188. SIAM (2005)
- [30] Kuperberg, G.: Another subexponential-time quantum algorithm for the dihedral hidden subgroup problem. *arXiv preprint arXiv:1112.3333*. (2011)
- [31] Bonnetain, X., Schrottenloher, A.: Quantum security analysis of CSIDH. *EUROCRYPT 2020*, pp. 493–522. Springer (2020)

- [32] Peikert, C.: He gives C-sieves on the CSIDH. Annual international conference on the theory and applications of cryptographic techniques, pp. 463–492. Springer (2020)